```
    threadsafe_queue& operator=(const threadsafe_queue&
other)=delete;

    std::shared_ptr<T> try_pop()
    {
       std::unique_ptr<node> old_head=pop_head();
       return old_head?old_head->data:std::shared_ptr<T>();
    }

    void push(T new_value)
    {
        std::shared_ptr<T> new_data(
            std::make_shared<T>(std::move(new_value)));
        std::unique_ptr<node> p(new node);
        node* const new_tail=p.get();
        std::lock_guard<std::mutex> tail_lock(tail_mutex);
        tail->data=new_data;
        tail->next=std::move(p);
        tail=new_tail;
    }
};
```

Let's look at this code with a critical eye, thinking about the guidelines listed in section 6.1.1. Before you look for broken invariants, you should be sure what they are:

- `tail->next==nullptr`.
- `tail->data==nullptr`.
- `head==tail` implies an empty list.
- A single element list has `head->next==tail`.
- For each node `x` in the list, where `x!=tail`, `x->data` points to an instance of `T` and `x->next` points to the next node in the list. `x->next==tail` implies `x` is the last node in the list.
- Following the `next` nodes from `head` will eventually yield `tail`.

On its own, `push()` is straightforward: the only modifications to the data structure are protected by `tail_mutex`, and they uphold the invariant because the new tail node is an empty node and `data` and `next` are correctly set for the old tail node, which is now the last real node in the list.

The interesting part is `try_pop()`. It turns out that not only is the lock on `tail_mutex` necessary to protect the read of `tail` itself, but it's also